

```

import java.util.*;

public class QuickSort<T extends Comparable<T>> {
    // Java cannot handle generic types in arrays due to type erasure.
    // Proper arrays have type information stored at runtime
    // Need to use a different container that is type safe.
    private List<T> A;
    private final Comparator<T> comparator;

    public QuickSort(List<T> unorderedArray) {
        A = new ArrayList<>(unorderedArray);
        comparator = new Comparator<T>() {
            @Override
            public int compare(T o1, T o2) {
                return o1.compareTo(o2);
            }
        };
    }

    public List<T> Sort() {
        Sort(0, A.size()-1);
        return (List<T>) new ArrayList(A);
    }

    private void Sort(int p, int r) {
        if(p < r) {
            int q = Partition(p, r);
            Sort(p, q-1);
            Sort(q+1, r);
        }
        return;
    }

    private int Partition(int p, int r) {
        T x = A.get(r);
        int i = p-1;
        for(int j = p; j < r; j++) {
            int compareResult = comparator.compare(A.get(j), x);
            if(comparator.compare(A.get(j), x) < 0) {
                i++;
                Collections.swap(A, i, j);
            }
        }
        Collections.swap(A, i+1, r);
        return i+1;
    }
}

```